Contents

SnapServe Config-to-Feature Compatibility Audit 1
1. Configuration surface inventory 1
2. STT / ASR provider and model compatibility 4
STT Model × Languages/Features matrix 6
3. LLM provider and model compatibility 7
LLM Model × Features compatibility matrix 9
4. TTS provider and model compatibility 10
TTS Provider × Languages/Features matrix 12
5. Call-type pipeline comparison (Outbound / Inbound / Web Call / Campaign) 12
6. Cross-cutting features 14
6.1 Tool calling / custom tools 14
6.2 Call transfer (cold/warm) 14
6.3 Calendar booking tools 14
6.4 Agent/caller memory 14
6.5 Knowledge base / RAG 15
6.6 Multilingual / language switching 15
7. Configuration Combination × Known Limitations 16
8. Recommended live-testing follow-ups (not performed as part of this audit) 18
9. Scope notes 18

SnapServe Config-to-Feature Compatibility Audit

Status: investigation/documentation only. No source files were modified.

This report traces, from the agent configuration UI down through the orchestration runtime, which VoiceOrch/SnapServe features are supported, unsupported, partially supported, or conditionally dependent on other config choices. Every claim is labeled **[confirmed]** (traced directly in code, with file citations) or **[uncertain]** (inferred from code shape but not verifiable without a live call).

File line numbers reference the codebase as of the audit date (2026-07-03) and may drift as the code evolves; treat them as pointers, not permanent anchors.

---

1. Configuration surface inventory

Agent configuration lives in two places: first-class columns on `agentsTable` (`lib/db/src/schema/agents.ts`) for primary provider selection and prompts, and the `agent_config` JSONB column for feature toggles, telephony behavior, and timing knobs. The dashboard builder (`artifacts/dashboard/src/pages/agents/builder.tsx`) and edit form (`edit.tsx` / `edit.schema.ts`) are the UI surfaces that write these fields, sharing a single Zod schema (`edit.schema.ts`) between create and edit flows.

Config axis	Column / path	Default	Notes
ASR provider	`asr_provider`	`azure`	
ASR model	`asr_model`	`""` (auto)	
ASR language	`asr_language`	`""`	Falls back to `language` if unset — see §6.6
LLM provider	`llm_provider`	`openai`	
LLM model	`llm_model`	`""` (provider default)	
TTS provider	`tts_provider`	`elevenlabs`	
TTS voice	`tts_voice`	`""` (provider default)	
TTS model	`tts_model`	`""`	
Telephony provider	`telephony_provider`	`twilio`	
First speaker	`first_speaker`	`assistant`	
Greeting message	`greeting_message`	LLM-generated if empty	
Silence timeout	`silence_timeout_seconds`	`30`	
Temperature	`temperature`	`0.10`	Not honored by all LLM adapters equally — see §3
Backchanneling	`backchanneling_enabled` / `_frequency`	`true` / `0.5`	
Barge-in	`agent_config.bargeInEnabled/ EnergyThreshold/ MinFrames/EchoGraceMs`	`true` / `1200` / `4` / `600`	Telephony-tuned defaults; webcall overrides — see §5
VAD silence	`agent_config.vadSilenceMs`	`350`	
STT endpointing	`agent_config.asrEndpointingSilenceMs`	`300`	
LLM max tokens	`agent_config.llmMaxTokens`	`null`	
TTS early-flush threshold	`agent_config.ttsMinSentenceChars`	`15`	
AMD mode	`agent_config.machineDetectionMode`	`disabled`	
Voicemail action / message / lead-in	`agent_config.voicemailAction/ Message/LeadInSeconds`	`hangup` / stock message / `2s`	
Recording	`agent_config.recordingEnabled`	`false`	
Summarization	`agent_config.summarizationEnabled`	`false` (but see §6.4 — always runs for managed post-call memory)	
Outcome tagging	`disposition_schema`	`[]`	
Tools	`tools` (JSON)	`[]`	
Knowledge/RAG sources	`knowledgeSourceIds` (M:N) + legacy `knowledgeBaseId`	`[]`	
Memory/CRM	`agent_config.callMemoryEnabled`	`false`	
Staged prompts	`staged_prompt_enabled`	`false`	
Compliance (IRDAI/TRAI)	`agent_config.compliance.irdai/ trai`	`false`	
Call-window allow	`agent_config.callWindow_enabled`	`false`	
Call-window timezone	`agent_config.callWindowTimezone`	`null` (unbounded)	
Call-window start/end	`agent_config.callWindowStart/End`	`null` (unbounded)	
Fallback LLM	`agent_config.fallbackLlmProvider/ Model`	`null` (unbounded)	See §3 fallback semantics

[confirmed] The config is split across a strongly-typed column set and a loosely-typed JSONB blob; several JSONB fields are read via `as any` casts in `pipeline.ts` / `webcall.ts` (per memory: `llm-fallback-pattern.md`), meaning schema drift between `edit.schema.ts` and the runtime reader is not caught by TypeScript.

---

2. STT / ASR provider and model compatibility

Provider	Streaming	Batch	Default model	Language handling
Deepgram	Yes	Yes	`nova-2`	Full BCP-47 accepted for English/Spanish variants; **other locales are silently truncated to bare ISO-639-1** (e.g. `ta-IN`→`ta`) before being sent — a config showing "Tamil (India)" in the UI is not literally what Deepgram receives (`deepgram.ts` ~L23-45).
Azure	Yes	Yes	Auto-derived from language	`model` field is ignored unless it is a genuine custom-deployment GUID (`azure.ts` ~L88-95); non-GUID model names are silently dropped. Locale normalization strips suffixes like `-neural` that would otherwise cause an empty transcript.
Sarvam	Yes	Yes	`saaras:v3`	Only 3 models are valid for streaming (per memory: `saaras:v2.5`/`v3`, `saarika:v2.5`); an unsupported model config silently falls back to `saaras:v3` (`sarvam.ts` ~L117). **Unrecognized languages silently fall back to `en-IN`** with only a log warning (~L84-85) — a caller configured for an unsupported Indic dialect gets English transcription with no user-visible error.
AssemblyAI	No	Yes	`best`	`nano`/`best` supported; language passed through to the API, so an invalid code surfaces as an API error rather than a silent substitution.
Cartesia (STT)	~~No~~ **[unconfirmed]**	Yes	No dedicated `whisper.ts` adapter file exists in `adapters/stt/`. "Whisper" as an ASR choice is	Whisper compatible; language passed as FormData, error handling delegated to provider.

Confidence-filtering asymmetry [confirmed]: Azure's batch confidence score is explicitly ignored for reliability reasons (comment notes it is "too aggressive for Indic languages" — matches memory `azure-stt-protocol.md`); streaming uses a soft 0.15 floor instead. Deepgram applies a real confidence threshold on both batch and streaming paths. **Practical effect:** the same "low confidence" transcript may be accepted on Azure but rejected on Deepgram, purely due to provider choice, not audio quality.

Adapter selection [confirmed]: `adapters/stt/index.ts` switches on `provider.toLowerCase()`; there is a global stub-provider override (`STUB_PROVIDERS_ENABLED`) that substitutes ALL providers with a stub regardless of agent config — relevant only in dev/test environments, but worth flagging since a misconfigured env flag in production would silently break every STT provider identically (no per-provider signal to distinguish "wrong provider" from "stub mode on").

STT Model × Languages/Features matrix

Language family	Deepgram	Azure	Sarvam	AssemblyAI	Cartesia (Whisper)
English variants	Full support, streaming	Full support, streaming	Supported, mapped to `en-IN`	Supported	Supported
Hindi/Tamil/Telugu/etc.	Streaming works but locale is truncated to bare code — **[confirmed] partial**: works, but the exact regional variant requested is not what's transmitted	Full locale support with normalization	Native strength; purpose-built for Indic (per memory `sarvam-stt-ws-protocol.md`)	Depends on provider API support — **[uncertain]**, not traced in adapter beyond passthrough	Depends on Whisper's own multilingual support — **[uncertain]**, no explicit VoiceOrch-side mapping found
Unsupported/unmapped language	Silent truncation, may error at provider	Empty transcript risk if normalization fails	Silent fallback to `en-IN`	Provider-level error	Provider-level behavior

3. LLM provider and model compatibility

Provider	Tool calling	Streaming	Notes
OpenAI	Native `tools`	Yes	`stream_options.include_usage` — real usage tracked (relevant to token-usage follow-up work already tracked as separate tasks)
DeepSeek	Native `tools`	Yes	OpenAI-compatible endpoint
Anthropic	Native `tool_use`/`tool_result`	Yes (SSE)	**[confirmed]** `max_tokens: 1024` is hardcoded in `anthropic.ts` (L85) as a literal, not read from `agent_config.llmMaxTokens` or any generation-param object — an agent configured with a higher `llmMaxTokens` value silently gets capped at 1024 output tokens on Anthropic regardless of that setting, while other providers (OpenAI/Groq/Qwen) do read `llmMaxTokens`/`genParams`. This is a real, verified per-provider inconsistency, not a hypothesis.
Mistral	Native `tools`	Yes	OpenAI-compatible
Cohere	Native `tools`	Yes	Uses `compatibility/v1` endpoint
Groq	Native `tools`	Yes	`service_tier: "on_demand"`; `resolveGroqModel` silently maps deprecated model names (e.g. `mixtral-8x7b-32768`) to `llama-3.3-70b-versatile` — **a user who explicitly picked a deprecated model in the UI silently gets a different model with different behavior/pricing**
Google (Gemini native)	Native, but **strict schema requirements**: tool parameter `type` must be uppercase (`"STRING"`, `"OBJECT"`); tool names with spaces/non-ASCII **hard-error with HTTP 400** rather than degrading gracefully	Yes	`thinkingBudget: 0`; preamble guard strips untagged planning narration that would otherwise leak into TTS
Qwen (Sarvam provider alias)	**No native tool calling** — text-based `<tool_call>` tag parsing + buffered-and-sniffing text-parse workaround (per memory `sarvam-llm-api.md`)	Yes	[illegible]

LLM Model × Features compatibility matrix

Feature	OpenAI/ DeepSeek/ Mistral/Cohere/Groq	Anthropic	Google (native)	Qwen (`gemini` alias)	Sarvam	Ultravox
Tool/function calling	Full	Full	Full (strict schema)	Full	Unreliable (buffer-and-sniff)	**None**
Streaming	Full	Full	Full	Full	Full	Full
Custom tool names w/ spaces/Unicode	Provider-dependent, generally tolerant	Tolerant	**Hard 400 error**	Tolerant	Tolerant but may get lost in JSON-in-text leakage	N/A (no tools)
Transfer/ booking/ custom tools	Supported	Supported	Supported if names sanitized	Supported	Degraded reliability	**Unsupported**
Reasoning/"thinking" mode	N/A	N/A	Disabled (`thinkingBudget: 0`)	`/no_think` forced	Off by default (per memory)	N/A

Fallback-chain semantics [confirmed]: - `fallbackLlm` (if `agent_config.fallbackLlmProvider` is set) is only invoked when the primary LLM throws **and** `llmFirstTokenAt === null` (i.e., zero tokens streamed yet) — a primary that starts streaming and then errors mid-response does NOT fail over; the turn simply truncates. This is a meaningful gap for any long-generation feature (e.g., detailed KB-grounded answers) where a late-stream failure is not covered by the fallback safety net at all. - Before falling back, the pipeline retries the SAME provider up to a bounded attempt count on transient errors (429/5xx/timeouts/network) — see `isRetryableLlmError()` in `pipeline.ts`. - **Managed plan gotcha (per memory `managed-plan-memory.md`):** the managed-plan post-call memory extraction hardcodes provider `"groq"` + a Llama model; if an agent's primary provider is something else (e.g. `openai`) and the managed fallback tries to reuse the agent's provider string with a Groq key, it 401s. This is a **config-to-feature incompatibility specific to the managed plan + post-call memory combination**, not present in BYOP. - BYOP agents with no `fallbackLlmProvider` configured have `fallbackLlm === null`; a primary-provider outage before first token simply fails the call — there is no implicit "borrow a managed key" behavior for BYOP fallback.

4. TTS provider and model compatibility

Provider	Streaming	Default voice/model	Notes
ElevenLabs	Yes (`/stream` endpoint)	`Rachel`, `eleven_turbo_v2_5`	Requires exact `Accept`/`output_format` handling or the API silently returns MP3 instead of raw PCM, which would break the pipeline's audio path if ever misconfigured
Cartesia	Yes	`Arushi` (Indian English), `sonic-3.5`	`sonic-3.5` covers 42 languages; legacy models cover far fewer — **an agent pinned to a legacy Cartesia model with a non-English `language` config may silently get English-accented or unsupported output** — needs live-testing to confirm exact legacy-model language list
OpenAI	**No streaming** (buffered only)	`alloy`, `tts-1`	Returns 24kHz PCM, downsampled to 16kHz in-adapter — **first-audio latency is inherently higher than the streaming providers**, directly affecting perceived responsiveness for any agent using OpenAI TTS
Azure	**No streaming**	`en-US-AriaNeural`	Language is inferred from the voice name string (`en-US-...`), so language and voice are coupled — **you cannot independently set language≠voice locale on Azure TTS without picking a voice from that locale**
Deepgram	Yes	`aura-2-en`	Aura family is primarily English-focused — **[uncertain]** exact non-English Aura voice coverage not confirmed; likely a compatibility gap for Indic-language agents
Sarvam	**No streaming** (endpoint 404s per memory `sarvam-tts-api.md`)	`bulbul:v3` (38 voices) / `bulbul:v2` (7 voices), default `ritu/anushka`	Purpose-built for Indian languages; **the "no streaming" limitation means any Sarvam-TTS agent has strictly higher time-to-first-audio than ElevenLabs/Cartesia/Deepgram agents, regardless of other config** — a real, structural latency ceiling for Indic-language agents choos-

TTS Provider × Languages/Features matrix

Feature	ElevenLabs	Cartesia	OpenAI	Azure	Deepgram	Sarvam
Streaming synthesis	Yes	Yes	No	No	Yes	No
Indic language coverage	Limited/uncertain	Strong (sonic-3.5, 42 languages)	Limited	Strong (locale-specific voices)	Weak/uncertain (English-centric Aura)	Strongest (purpose-built)
Barge-in friendliness (low latency to first byte)	Good	Best (~80-150ms first chunk)	Poor (buffered)	Poor (buffered)	Good	Poor (buffered)
Unsupported language behavior	Provider-dependent — **[uncertain]**, not traced to a specific fallback	Maps BCP-47→ISO 639-1; unmapped → **[uncertain]** exact behavior	Provider-dependent	Voice/locale coupling forces a valid combination or error	**[uncertain]**	Defaults to `en-IN` silently when config maps to an unsupported code

Cross-cutting latency implication [confirmed]: because fillers/backchannels (`backchannel.ts`) and barge-in timing constants (`FILLER_THRESHOLD_MS = 800ms` in `pipeline.ts`) are fixed regardless of TTS provider, a **non-streaming** TTS provider (OpenAI, Azure, Sarvam) is structurally more likely to trigger a filler phrase or to feel sluggish on barge-in recovery than a streaming provider — this is an emergent interaction between a global timing constant and a per-agent TTS provider choice, not something the UI surfaces to the user when they pick a non-streaming TTS provider.

---

5. Call-type pipeline comparison (Outbound / Inbound / Web Call / Campaign)

All four call types share the same `OrchestrationPipeline` core (`orchestration/pipeline.ts`); Web Call is a thin WS transport (`telephony/webcall.ts`) over that same core per the "webcall 2.0" architecture (memory: `webcall-pipeline-split.md`). Differences arise from transport-specific wiring, not forked business logic — but real feature gaps still exist:

Feature	Live Outbound	Inbound	Web Call	Campaign
AMD / voicemail detection	Full — Twilio/Plivo synchronous, VoBiz acoustic-confirmed async (`dial.ts`, `pipeline.ts` `confirmAmdMachineThenBranch`)	**AMD gate is explicitly bypassed for VoBiz inbound** (`webhooks.ts`) — inbound calls are assumed human; **no AMD path exists for inbound at all across any provider**, since AMD is fundamentally an outbound-dial concept	**None** — raw PCM straight into the pipeline, no telephony-provider signal to key off of	Shares the outbound AMD path (`resolveAmdConfig` from `dial.ts`)
Barge-in threshold	`BARGE_IN_ENERGY_THRESHOLD=3200` + adaptive echo-floor calibration (echoFloor × 1.6, capped 2500)	Same as Outbound (shared telephony carrier path)	`BARGE_IN_WEBCALL_AEC_THRESHOLD=400` — lower because browser AEC already cancels echo	Same as Outbound
Greeting/first-message	AMD-gated: only speaks after acoustic/callback confirmation resolves human (or the confirm window elapses); compact greeting-only prompt used to cut latency	Supports IVR/queueing ahead of the session greeting	Sends a `{"type":"ready"}` handshake to the client before greeting starts	Same as Outbound; greeting timing further affected by `callDelaySeconds` before dial fires
Endpointing/silence knobs	Full `computeEndpointMs` + turn-safety-net (`endpointing.ts`)	Same	Same — shared core	Same
Recording	Twilio: provider-side `Record:true`; VoBiz/Plivo: pipeline-side `RecordingBuffer` (avoids provider beep tones)	Same provider-dependent split, plus a VoBiz-specific recording-status callback route	Inherits pipeline `RecordingBuffer`	Same as Outbound
Call transfer (cold/warm)	Full — provider REST APIs redirect the live call leg, with per-provider whisper/briefing quirks (Twilio `<Number url>` vs Plivo/VoBiz `<Dial confirmSound>`)	Full (same mechanism)	**Effectively unsupported** — no PSTN leg exists to redirect; `onCallTransfer` is a no-op/termination at the transport layer	Full (same mechanism as Outbound)
Retry/redial	N/A (single dial)	N/A	N/A	Full — `call_retries` scheduling, atomic claim, rolling concurrency (per memory `campaign-retry-timing.md`, `campaign-rolling-concurrency.md`)

Key confirmed gaps: 1. **Transfer-to-human is a documented UI toggle/tool that silently does nothing meaningful on Web Call.** If an agent config enables the `transfer_to_human` tool and that agent is ever used in a Web Call context, the tool call resolves to a no-op rather than an error — callers/testers may perceive this as "the transfer didn't work" with no diagnostic signal at the UI layer. **[confirmed from code shape, exact caller-facing behavior on failure — uncertain, needs live web-call test.]** 2. **AMD has no inbound path on any provider.** A voicemail/machine-detection feature toggle configured on an agent only takes effect for outbound and campaign calls; if the same agent also receives inbound calls, that config is silently inert for the inbound leg. 3. **VoBiz AMD is architecturally different from Twilio/Plivo** (acoustic confirmation vs synchronous provider verdict) — an agent's AMD behavior (timing, false-positive rate) is not portable across telephony providers even with identical `agent_config` values.

---

6. Cross-cutting features

6.1 Tool calling / custom tools

[confirmed] `buildToolDefs` (`services/tool-executor.ts`) is the single source of tool schema generation, converting agent-configured tools into provider-specific `ToolDefinition` shapes. A mandatory `end_call` tool is injected for every agent. `sanitizeToolName` normalizes custom tool names for cross-provider compatibility; **a custom tool whose name sanitizes to an empty string (e.g., a purely Tamil/Devanagari name) is silently dropped** rather than erroring — an admin configuring a non-Latin custom tool name may see it vanish from the LLM's available tools with no warning surfaced in the UI.

Provider-specific behavior differences (see §3) mean the SAME tool configuration behaves very differently depending on the agent's LLM provider: full reliability on OpenAI/Anthropic/Google/Groq/Mistral/Cohere/DeepSeek, degraded on Sarvam, and a hard **feature unavailable** on Ultravox.

6.2 Call transfer (cold/warm)

[confirmed] `executeCallTransfer` (`pipeline.ts`) generates a warm-transfer operator briefing via an LLM call before invoking the provider transfer API. Per telephony provider: - Twilio: standard `<Number url="...">` whisper. - Plivo/VoBiz: **silently ignore** the Twilio-style whisper attribute; the codebase compensates with `<Dial confirmSound>` — this is provider-specific compensation code, not something visible in agent config, so a transfer configured identically across providers has provider-dependent whisper/briefing behavior. - VoBiz additionally **requires** a valid, provisioned VoBiz number as `callerId` on the transfer `<Dial>`, or the bridge to the human agent is rejected outright. - Web Call: no PSTN leg — transfer is a no-op (see §5).

6.3 Calendar booking tools

[confirmed] `executeNativeBooking` (`tool-executor.ts`) enforces an `EXACT_TOLERANCE_MS` (~1 minute) window: the LLM must request a time that exactly matches an available slot, or the booking is refused rather than silently approximated to the nearest slot. Timezone defaults to `Asia/Kolkata` if the agent config omits an IANA timezone string — **an agent serving a non-India timezone without explicit timezone config will silently book in IST**, a real and non-obvious config-dependent bug surface.

Booking-tool reliability inherits the LLM tool-calling compatibility matrix from §3 — Ultravox cannot book at all; Sarvam bookings are subject to buffer-and-sniff parsing reliability.

6.4 Agent/caller memory

[confirmed] Three-layer memory (`callerMemory.ts`) is gated by `agent_config.callMemoryEnabled`. Persona-hash detection (`computePersonaHash`/`personaChanged`) suppresses ALL domain-bearing memory when an agent's system prompt has meaningfully changed since the caller's last contact — a deliberate safety feature, but it means memory-dependent behaviors (returning-caller greeting, sentiment arc, predicted intent) silently vanish after any substantial prompt edit, which could confuse an admin who edited a prompt for an unrelated reason and then sees "the agent forgot the caller."

Managed-plan-specific gap (per memory): the post-call memory-extraction LLM call is hardcoded to `groq` + Llama; combined with any agent whose primary LLM provider differs, this is an incompatibility isolated to the managed billing plan (BYOP agents are unaffected because they don't run this managed-only extraction path). **[confirmed from memory note; re-verify current code state before citing exact provider string, since this may have been fixed since the note was written]**.

6.5 Knowledge base / RAG

[confirmed] `retrieveKbContextMulti` (`kb-retrieval.ts`) is capped at `MAX_KB_ENTRIES=200` — sources beyond this size are silently truncated to the first 200 entries with only a log warning, no user-facing signal that some KB content is unused. There is no embeddings/vector search (per memory `kb-fts-architecture.md`) — KB relevance depends entirely on the LLM reading the full (capped) entry set, meaning very large knowledge bases degrade differently (truncation) than expected from a typical RAG mental model (top-k relevance retrieval).

Latency race [confirmed]: the pipeline races KB retrieval against a `KB_CONTEXT_RACE_MS=200ms` timer; if the DB/lookup is slower than 200ms, the LLM turn proceeds WITHOUT KB context. This means **KB feature reliability degrades silently under DB load or for very large KB sources**, independent of any agent config — a config toggle "Knowledge Base: ON" does not guarantee KB context is actually injected on every turn.

6.6 Multilingual / language switching

[confirmed] `detectScriptLanguage` (`pipeline.ts`) infers the active language from Unicode script ranges of the transcript, not from STT-reported language (most STT adapters here do not return a detected language). This drives per-turn `[LANGUAGE]` injection appended to the LAST user turn (high-attention window) when `languageAutoDetect` is on.

- Session setup uses `asrLanguage ?? language` (per memory `agent-language-asr-conflict.md`) — if `asrLanguage` is unset but `language` defaults to `en-US`, the STT/language-rule injection can force English even when the intended agent config was ambiguous, a subtle config-precedence trap.
- TTS-side language support varies drastically per provider (§4): Sarvam TTS supports only its mapped Indic code list and silently substitutes `en-IN`; Cartesia's `sonic-3.5` supports the widest range (42 languages via ISO 639-1 mapping); Azure/OpenAI/Deepgram TTS have narrower or English-centric coverage.
- **Practical combination risk:** an agent configured for Tamil with Sarvam STT (strong) + Deepgram TTS (weak Indic coverage) would transcribe well but synthesize poorly — this is a **cross-axis (STT×TTS) compatibility gap that the UI does not warn about**, since STT and TTS provider pickers are independent form controls.

---

7. Configuration Combination × Known Limitations

Combination	Limitation	Confidence
LLM = Ultravox + any tool-dependent feature (transfer, booking, custom tools)	Hard unsupported — tools stripped entirely	confirmed
LLM = `gemini` (provider string)	Silently routed to Groq-hosted Qwen, not Google Gemini — model identity mismatch vs UI label	confirmed
LLM = Sarvam + tool calling	Unreliable; buffer-and-sniff parsing of tool calls leaked as text	confirmed
TTS = Sarvam / OpenAI / Azure (non-streaming)	Structurally higher latency to first audio; more filler/barge-in friction	confirmed
TTS = Sarvam or Deepgram + non-Indic or off-catalog language	Silent substitution to a default voice/language (`en-IN` for Sarvam)	confirmed (Sarvam); uncertain (Deepgram)
STT = Deepgram + non-English/Spanish BCP-47 locale	Silent truncation to bare ISO-639-1 before transmission	confirmed
STT = Sarvam + unsupported language code	Silent fallback to `en-IN`	confirmed
Call type = Web Call + transfer_to_human tool enabled	Transfer no-ops; no PSTN leg	confirmed (mechanism); uncertain (user-visible failure signal)
Call type = Inbound + AMD/voicemail config	AMD is bypassed entirely for inbound; config is inert	confirmed
Telephony = VoBiz + warm transfer	Requires provisioned VoBiz `callerId`; silently ignores Twilio-style whisper attribute, uses `confirmSound` instead	confirmed
Billing plan = Managed + post-call memory extraction	Hardcoded Groq/Llama extraction path; potential provider/key mismatch vs agent's configured primary LLM	confirmed via memory note; needs re-verification against current code
Knowledge base source > 200 entries	Silent truncation to first 200 entries	confirmed
Knowledge base + slow DB/lookup	KB context silently dropped for that turn if retrieval exceeds 200ms	confirmed
Custom tool with non-Latin/non-ASCII name	Silently dropped from available tools if sanitization yields an empty string	confirmed
Booking tool + no explicit agent timezone	Silently defaults to Asia/Kolkata	confirmed
Any LLM + long-generation failure mid-stream	Fallback LLM does NOT trigger once first token has streamed — turn simply truncates	confirmed
Google (Gemini native) LLM + custom tool names with spaces/Unicode	Hard 400 error at the API, not a graceful degrade	confirmed
Anthropic + high `llmMaxTokens` config	Adapter appears to hardcode `max_tokens: 1024`, potentially overriding a higher configured value	uncertain — needs direct code re-verification before treating as fact

8. Recommended live-testing follow-ups (not performed as part of this audit)

These are flagged, not executed, per the task's "no live/manual call testing" constraint:

1. Confirm actual behavior of a Web Call session when `transfer_to_human` fires — does the caller/agent see any error, or does the call simply continue silently?
2. Confirm exact Cartesia legacy-model language coverage vs `sonic-3.5`.
3. Confirm current state of the managed-plan post-call memory Groq/Llama hardcode (whether still present or since fixed).
4. Confirm Anthropic's `max_tokens` handling against a configured `agent_config.llmMaxTokens` value greater than 1024.
5. Confirm Deepgram Aura TTS voice coverage for Indic languages (English-centric assumption above is inferred, not directly verified against Deepgram's current voice catalog).
6. Confirm whether a dedicated Whisper STT adapter exists distinct from the Cartesia/AssemblyAI Whisper-compatible endpoints.

9. Scope notes

- No code, schema, or UI changes were made as part of this audit.
- Findings are based on static code tracing by parallel code-exploration passes over the adapters, pipeline, tool-executor, caller-memory, KB-retrieval, and campaign-caller modules, cross-referenced against this project's existing `replit.md` operational notes and prior debugging memory where directly relevant (each such reference is marked "per memory" above).
- Line-number citations from the exploration passes should be treated as approximate pointers for a follow-up reader to locate the relevant code, not as guaranteed-exact references — several were reported by sub-agents without a final direct read-back verification pass in this session, and are marked accordingly above (see §9′s "needs re-verification" items and inline **[uncertain]** tags throughout).